Buổi 10 (3 tiết) – Chương 5: APIs, Debug, Xuất bản + Lab cuối (5.1–5.8)

1 Chương 5: Lập trình ứng dụng với APIs

1.1 5.1 Sử dụng thư viện Geolocation

Geolocation là một tính năng quan trọng cho phép ứng dụng di động truy cập vị trí địa lý của người dùng. React Native cung cấp một API tích hợp để tương tác với các dịch vụ vị trí của thiết bị.

1.1.1 Bắt tọa độ người dùng

Để lấy tọa độ hiện tại của người dùng, bạn có thể sử dụng hàm Geolocation.getCurrentPosition(). Hàm này nhận ba đối số:

- **successCallback:** Hàm được gọi khi lấy vị trí thành công. Nó nhận một đối tượng Position chứa thông tin về vị trí (kinh độ, vĩ độ, độ chính xác, độ cao, tốc độ, v.v.).
- **errorCallback (tùy chọn):** Hàm được gọi khi xảy ra lỗi trong quá trình lấy vị trí. Nó nhận một đối tượng PositionError chứa thông tin về lỗi.
- **options (tùy chọn):** Một đối tượng chứa các tùy chọn cấu hình cho việc lấy vị trí (ví dụ: độ chính xác cao, thời gian chờ tối đa, thời gian hết hạn).

Ví dụ:

``javascript import React, { useEffect, useState } from 'react'; import { View, Text, StyleSheet, PermissionsAndroid,Platform } from 'react-native'; import Geolocation from 'react-native-geolocation-service';

const LocationScreen = () => { const [location, setLocation] = useState(null); const [error, setError] = useState(null);

useEffect(() => { const requestLocationPermission = async () => { let granted = true; if (Platform.OS === 'android') { granted = await PermissionsAndroid.request(PermissionsAndroid.PERMISSIONS.ACCESS_FINE_LOCATION, { title: 'Yêu cầu quyền truy cập vị trí', message: 'Ứng dụng cần quyền truy cập vị trí của bạn ' + 'để cung cấp các tính năng liên quan đến vị trí.', buttonNeutral: 'Để sau', buttonNegative: 'Hủy', buttonPositive: 'OK', },); if (granted === PermissionsAndroid.RESULTS.GRANTED) { console.log('Bạn đã cấp quyền truy cập vị trí'); } else { console.log('Quyền truy cập vị trí bị từ chối'); setError('Quyền truy cập vị trí bị từ chối'); } }

```
if (granted) {
  Geolocation.getCurrentPosition(
    (position) => {
      console.log(position);
      setLocation(position.coords);
      setError(null);
    },
    (err) => {
      setError(err.message);
    },
    { enableHighAccuracy: true, timeout: 15000, maximumAge: 10000 }
  );
```

```
  }

};

requestLocationPermission();

}, []);

return ( &lt;View style={styles.container}> &lt;Text style={styles.title}>Thông tin vị trí&lt;/Text> {location ? ( &lt;View> &lt;Text>Kinh độ: {location.longitude}&lt;/Text> &lt;Text>Vĩ độ: {location.latitude}&lt;/Text> &lt;Text>Độ chính xác: {location.accuracy}&lt;/Text> &lt;/View> ) : ( &lt;Text>{error || 'Đang lấy vị trí...'}&lt;/Text> )} &lt;/View> ); };

const styles = StyleSheet.create({ container: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20, }, title:{ fontSize: 20, fontWeight: 'bold', marginBottom: 15, }, });

export default LocationScreen; ``
```

1.1.2 Quản lý phân quyền

Trước khi có thể truy cập vị trí của người dùng, ứng dụng cần phải được cấp quyền truy cập vị trí từ hệ điều hành.Cách quản lý phân quyền khác nhau giữa Android và iOS:

- **Android:** Bạn có thể sử dụng PermissionsAndroid.request() để yêu cầu quyền truy cập vị trí tại thời điểm chạy. Bạn nên giải thích rõ ràng lý do ứng dụng cần quyền này cho người dùng.
- **iOS:** Quyền truy cập vị trí được quản lý tự động bởi hệ thống khi bạn sử dụng các API liên quan đến vị trí.Người dùng sẽ được hỏi quyền khi ứng dụng cố gắng truy cập vị trí lần đầu tiên. Bạn có thể cấu hình các thông báo giải thích quyền trong file Info.plist.

Trong ví dụ trên, chúng ta đã sử dụng PermissionsAndroid.request() cho Android để yêu cầu quyền ACCESS_FINE_LOCATION.

1.1.3 Bắt sự kiện và theo dõi tọa độ người dùng

Đôi khi bạn cần theo dõi vị trí của người dùng liên tục khi họ di chuyển (ví dụ: trong ứng dụng điều hướng hoặc theo dõi hoạt động thể thao). Bạn có thể sử dụng hàm Geolocation.watchPosition() để thiết lập một trình theo dõi vị trí.

Hàm này cũng nhận các đối số tương tự như Geolocation.getCurrentPosition(), nhưng successCallback sẽ được gọi mỗi khi vị trí của người dùng thay đổi. Geolocation.watchPosition() trả về một ID theo dõi mà bạn có thể sử dụng để dừng việc theo dõi sau này bằng cách gọi Geolocation.clearWatch(watchId).

Ví dụ:

``javascript import React, { useEffect, useState } from 'react'; import { View, Text, StyleSheet } from 'react-native'; import Geolocation from 'react-native-geolocation-service';

const TrackingLocationScreen = () => { const [location, setLocation] = useState(null); const [error, setError] = useState(null); const [watchId, setWatchId] = useState(null);

useEffect(() => { const startLocationTracking = async () => { // (Yêu cầu quyền tương tự như ví dụ trước) const granted = await // ... (Code yêu cầu quyền)

```
if (granted) {
  const id = Geolocation.watchPosition(
    (position) => {
      console.log('Vị trí thay đổi:', position);
      setLocation(position.coords);
      setError(null);
```

```
      },
      (err) => {
        setError(err.message);
      },
      { enableHighAccuracy: true, distanceFilter: 10, interval: 5000, fastestInterval: 2000 }
    );
    setWatchId(id);
  }
};

startLocationTracking();

return () => {
  // Dừng theo dõi khi component bị unmount
  if (watchId) {
    Geolocation.clearWatch(watchId);
  }
};
}, []);

return ( &lt;View style={styles.container}> &lt;Text style={styles.title}>Theo dõi vị trí&lt;/Text> {location ? ( &lt;View> &lt;Text>Kinh độ: {location.longitude}&lt;/Text> &lt;Text>Vĩ độ: {location.latitude}&lt;/Text> &lt;/View> ) : ( &lt;Text>{error || 'Đang theo dõi vị trí...'}&lt;/Text> )} &lt;/View> );
};
```

const styles = StyleSheet.create({ container: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20, }, title:{ fontSize: 20, fontWeight: 'bold', marginBottom: 15, }, });

export default TrackingLocationScreen; ``

1.2 5.2 Truy cập tài nguyên của thiết bị

React Native cung cấp các API để truy cập một số tài nguyên của thiết bị như thư viện ảnh và camera.

1.2.1 Sử dụng CameraRoll (đã lỗi thời)

Lưu ý: Module CameraRoll đã được tách ra khỏi React Native core và hiện nằm trong thư viện cộng đồng @react-native-camera-roll/camera-roll. Bạn cần cài đặt thư viện này để sử dụng.

CameraRoll cho phép bạn truy cập ảnh và video từ thư viện của thiết bị. Bạn có thể lấy danh sách các asset, lưu ảnh hoặc video vào thư viện.

Ví dụ lấy danh sách ảnh:

``javascript import React, { useState, useEffect } from 'react'; import { View, FlatList, Image, StyleSheet, Button, PermissionsAndroid, Platform } from 'react-native'; import CameraRoll from '@react-native-camera-roll/camera-roll';

const GalleryScreen = () => { const [photos, setPhotos] = useState([]); const [error, setError] = useState(null);

useEffect(() => { const getPhotos = async () => { let granted = true; if (Platform.OS === 'android') { granted = await PermissionsAndroid.request(PermissionsAndroid.PERMISSIONS.READ_EXTERNAL_STORAGE, { title: 'Yêu cầu quyền truy cập bộ nhớ', message: 'Ứng dụng cần quyền truy cập bộ nhớ để xem

ảnh.', buttonNeutral: 'Để sau', buttonNegative: 'Hủy', buttonPositive: 'OK', },); if (granted !== PermissionsAndroid.RESULTS.GRANTED) { setError('Quyền truy cập bộ nhớ bị từ chối'); return; } }

```
if (granted) {
  CameraRoll.getPhotos({
    first: 20, // Số lượng ảnh muốn lấy
    assetType: 'Photos', // Lọc theo loại asset (Photos, Videos, All)
  })
    .then(r => {
      setPhotos(r.edges);
    })
    .catch((err) => {
      setError('Không thể truy cập thư viện ảnh.');
      console.error(err);
    });
  }
};

getPhotos();
}, []);
```

if (error) { return (<View style={styles.container}> <Text>{error}</Text> </View>); }

return (<View style={styles.container}> <FlatList data={photos} keyExtractor={(item) => item.node.image.uri} renderItem={({ item }) => (<Image style={styles.image} source={{ uri: item.node.image.uri }} />)} numColumns={3} /> </View>); };

const styles = StyleSheet.create({ container: { flex: 1, }, image: { width: 120, height: 120, margin: 2, }, });

export default GalleryScreen; ``

1.2.2 Truy cập thư viện ảnh với Image Picker

Một thư viện phổ biến và được khuyến nghị để chọn ảnh và video từ thư viện hoặc chụp ảnh/quay video là react-native-image-picker. Bạn cần cài đặt thư viện này:

``bash npm install react-native-image-picker --save

hoặc

yarn add react-native-image-picker ``

Ví dụ chọn ảnh từ thư viện:

``javascript import React, { useState } from 'react'; import { View, Button, Image, StyleSheet } from 'react-native'; import { launchImageLibrary } from 'react-native-image-picker';

const ImagePickerScreen = () => { const [imageUri, setImageUri] = useState(null);

const selectImage = async () => { const result = await launchImageLibrary({ mediaType: 'photo' });

if (!result.didCancel && result.assets && result.assets.length > 0) {

setImageUri(result.assets.length > 0 ? result.assets?.[0]?.uri : null);

}

};

return (<View style={styles.container}> <Button title="Chọn ảnh từ thư viện" onPress={selectImage} /> {imageUri && (<Image source={{ uri: imageUri }} style={styles.image} />)} </View>); };

const styles = StyleSheet.create({ container: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20, }, image: { width: 200, height: 200, marginTop: 20, }, });

export default ImagePickerScreen; ``

1.2.3 Sử dụng Image Picker để chụp ảnh

Tương tự như chọn ảnh từ thư viện, bạn có thể sử dụng react-native-image-picker để mở camera và chụp ảnh.

Ví dụ chụp ảnh bằng camera:

``javascript import React, { useState } from 'react'; import { View, Button, Image, StyleSheet } from 'react-native'; import { launchCamera } from 'react-native-image-picker';

const CameraScreen = () => { const [imageUri, setImageUri] = useState(null);

const takePicture = async () => { const result = await launchCamera({ mediaType: 'photo', saveToPhotos: true });

if (!result.didCancel && result.assets && result.assets.length > 0) {

setImageUri(result.assets.length > 0 ? result.assets?.[0]?.uri : null);

}

};

return (<View style={styles.container}> <Button title="Chụp ảnh" onPress={takePicture} /> {imageUri && (<Image source={{ uri: imageUri }} style={styles.image} />)} </View>); };

const styles = StyleSheet.create({ container: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20, }, image: { width: 200, height: 200, marginTop: 20, }, });

export default CameraScreen; ``

1.3 5.3 Lưu trữ dữ liệu với AsyncStorage

AsyncStorage là một hệ thống lưu trữ key-value đơn giản, bất đồng bộ và không mã hóa. Nó thường được sử dụng để lưu trữ một lượng nhỏ dữ liệu mà không cần cơ sở dữ liệu quan hệ. Dữ liệu được lưu trữ là persistent giữa các lần khởi chạy ứng dụng.

Các thao tác cơ bản với AsyncStorage:

- **setItem(key, value):** Lưu một cặp key-value vào bộ nhớ. value phải là một string.
- **getItem(key):** Lấy giá trị được liên kết với key. Trả về một Promise giải quyết với giá trị (hoặc null nếu không tìm thấy).
- **removeItem(key):** Xóa mục được liên kết với key. Trả về một Promise.
- **clear():** Xóa tất cả các mục khỏi bộ nhớ. Trả về một Promise.
- **getAllKeys():** Lấy một mảng chứa tất cả các key trong bộ nhớ. Trả về một Promise.
- **multiSet(keyValuePairs):** Lưu nhiều cặp key-value cùng một lúc. keyValuePairs là một mảng các mảng ['key', 'value']. Trả về một Promise.
- **multiGet(keys):** Lấy giá trị cho nhiều key cùng một lúc. Trả về một Promise giải quyết với một mảng các mảng ['key', 'value'].
- **multiRemove(keys):** Xóa nhiều mục cùng một lúc. keys là một mảng các key. Trả về một Promise.

Ví dụ:

``javascript import React, { useState, useEffect } from 'react'; import { View, Text, TextInput, Button, StyleSheet } from 'react-native'; import AsyncStorage from '@react-native-async-storage/async-storage';

const AsyncStorageScreen = () => { const [inputValue, setInputValue] = useState(''); const [storedValue, setStoredValue] = useState(null); const [error, setError] = useState(null);

useEffect(() => { loadStoredValue(); }, []);

const loadStoredValue = async () => { try { const value = await AsyncStorage.getItem('myKey'); if (value !== null) { setStoredValue(value); } } catch (e) { setError('Lỗi khi tải dữ liệu.'); console.error(e); } };

const saveValue = async () => { try { await AsyncStorage.setItem('myKey', inputValue); setStoredValue(inputValue); setInputValue(''); setError(null); } catch (e) { setError('Lỗi khi lưu dữ liệu.'); console.error(e); } };

const clearValue = async () => { try { await AsyncStorage.removeItem('myKey'); setStoredValue(null); setError(null); } catch (e) { setError('Lỗi khi xóa dữ liệu.'); console.error(e); } };

return (<View style={styles.container}> <Text style={styles.title}>AsyncStorage Demo</Text> <TextInput style={styles.input} placeholder="Nhập giá trị để lưu" value={inputValue} onChangeText={text => setInputValue(text)} /> <Button title="Lưu" onPress={saveValue} /> <Text style={styles.label}>Giá trị đã lưu:</Text> <Text style={styles.value}>{storedValue !== null ? storedValue : 'Chưa có'}</Text> <Button title="Xóa" onPress={clearValue} /> {error && <Text style={styles.error}>{error}</Text>} </View>); };

const styles = StyleSheet.create({ container: { flex: 1, alignItems: 'center', justifyContent: 'center', padding: 20, }, title: { fontSize: 20, fontWeight: 'bold',

marginBottom: 20, }, input: { width: '80%', borderWidth: 1, borderColor: '#ccc', padding: 10, marginBottom: 10, }, label: { marginTop: 20, fontWeight: 'bold', }, value: { fontSize: 16, marginBottom: 10, }, error: { color: 'red', marginTop: 10, }, });

export default AsyncStorageScreen; ``

1.4 5.4 Debug trong React Native

Debug là một phần không thể thiếu trong quá trình phát triển ứng dụng React Native. Dưới đây là một số cách để debug ứng dụng của bạn:

1. **Sử dụng React Native Debugger:** Đây là một công cụ standalone mạnh mẽ kết hợp React Inspector và Redux DevTools, đồng thời cho phép bạn debug JavaScript từ xa. Bạn có thể tải xuống và cài đặt React Native Debugger từ trang GitHub của nó. Khi ứng dụng của bạn đang chạy ở chế độ debug, bạn có thể chọn "Debug Remote JS" trong developer menu (thường được mở bằng cách lắc thiết bị hoặc sử dụng phím tắt trên trình giả lập) để kết nối với debugger.

2. **Sử dụng Chrome Developer Tools:** Khi bạn chọn "Debug Remote JS", React Native sẽ mở một tab Chrome với một debugger. Bạn có thể sử dụng các công cụ quen thuộc của Chrome DevTools như Sources (để xem và chỉnh sửa code), Console (để xem logs và chạy JavaScript), Network (để theo dõi các yêu cầu mạng), và Performance (để phân tích hiệu suất).

3. **console.log():** Phương pháp đơn giản nhất để debug là chèn các lệnh console.log() vào code của bạn để theo dõi giá trị của các biến và luồng thực thi. Các log này sẽ xuất hiện trong Chrome DevTools console hoặc trong logcat (đối với Android) / system log (đối với iOS) nếu bạn đang chạy ứng dụng trực tiếp trên thiết bị.

4. **Breakpoints:** Bạn có thể đặt breakpoints trong Chrome DevTools (trong tab Sources) để tạm dừng việc thực thi code tại một điểm cụ thể và kiểm tra trạng thái của ứng dụng.

5. **React Inspector:** React Native Debugger tích hợp React Inspector, cho phép bạn kiểm tra cây component, props, và state của các component trong ứng dụng của mình một cách trực quan.

6. **Profiler:** React Developer Tools (có thể được sử dụng trong React Native Debugger hoặc như một extension của trình duyệt Chrome khi debug web build của React Native) cung cấp một profiler để phân tích hiệu suất render của các component.

7. **Sử dụng Flipper:** Flipper là một nền tảng debug có thể mở rộng cho các ứng dụng di động. Nó cung cấp nhiều plugin hữu ích cho React Native, bao gồm layout inspector, network inspector, crash reporter, và nhiều hơn nữa. Bạn cần cài đặt Flipper trên máy tính và tích hợp nó vào ứng dụng React Native của mình.

   Mẹo debug:

- Đảm bảo ứng dụng của bạn đang chạy ở chế độ debug.
- Kiểm tra kỹ các thông báo lỗi trong console. Chúng thường cung cấp thông tin hữu ích về vấn đề.
- Sử dụng breakpoints một cách chiến lược để hiểu rõ luồng thực thi của code.
- Tận dụng React Inspector để kiểm tra props và state của component khi có vấn đề về giao diện hoặc hành vi.

1.5 5.5 Xuất bản ứng dụng

Xuất bản ứng dụng React Native là quá trình tạo các bản dựng (builds) có thể cài đặt được cho các cửa hàng ứng dụng (Google Play Store cho Android và App Store cho iOS). Quá trình này bao gồm nhiều bước:

1. Cấu hình ứng dụng cho bản release:

- **Đổi tên bundle ID/package name:** Đảm bảo bundle ID (iOS) và package name (Android) là duy nhất và tuân theo quy ước.
- **Tạo signing key cho Android:** Bạn cần tạo một keystore file để ký APK/AAB của ứng dụng Android.
- **Cấu hình build scheme cho iOS:** Trong Xcode, bạn cần cấu hình scheme cho bản Archive.
- **Cập nhật icon và splash screen:** Đảm bảo ứng dụng có icon và splash screen chất lượng cao cho các kích thước màn hình khác nhau.
- **Gỡ bỏ các console logs và các code debug không cần thiết.**

2. Tạo bản dựng release:

- **Android:**
  - Chạy lệnh sau trong thư mục dự án: ``bash npx react-native bundle --platform android --dev false --entry-file index.js --bundle-output android/app/src/main/assets/index.android.bundle --assets-dest android/app/src/main/res ``
  - Sau đó, bạn có thể build APK hoặc AAB file bằng Android Studio (Build -> Generate Signed Bundle / APK).
- **iOS:**
  - Mở file ios/<YourAppName>.xcworkspace trong Xcode.
  - Chọn Generic iOS Device làm target build.
  - Chọn Product -> Archive để tạo file .ipa. Bạn sẽ cần có Apple Developer account và provisioning profiles/certificates phù hợp.

3. Kiểm tra bản dựng release:

- Cài đặt và kiểm tra kỹ lưỡng bản dựng release trên các thiết bị thực tế để đảm bảo ứng dụng hoạt động ổn định và không có lỗi.

4. Chuẩn bị thông tin xuất bản:

- **Tiêu đề và mô tả ứng dụng:** Viết tiêu đề hấp dẫn và mô tả chi tiết về ứng dụng.
- **Ảnh chụp màn hình và video giới thiệu:** Chuẩn bị các hình ảnh và video chất lượng cao để giới thiệu các tính năng của ứng dụng.
- **Chính sách bảo mật:** Cung cấp liên kết đến chính sách bảo mật của ứng dụng.
- **Thông tin liên hệ:** Cung cấp email hoặc website hỗ trợ.
- **Phân loại ứng dụng và độ tuổi:** Chọn phân loại phù hợp và chỉ định độ tuổi cho ứng dụng.

5. Gửi ứng dụng lên cửa hàng:

- **Google Play Store (Android):** Bạn cần tạo một tài khoản Google Play Console và làm theo quy trình tải ứng dụng lên. Bạn sẽ tải lên file APK hoặc AAB.
- **App Store (iOS):** Bạn cần có Apple Developer Program membership và sử dụng Xcode và Application Loader (hoặc Transporter) để tải file .ipa lên App Store Connect.

6. Quản lý ứng dụng sau khi xuất bản:

- Theo dõi hiệu suất ứng dụng và phản hồi của người dùng.
- Cập nhật ứng dụng thường xuyên với các tính năng mới và sửa lỗi.

Quá trình xuất bản ứng dụng có thể phức tạp và đòi hỏi sự chuẩn bị kỹ lưỡng. Hãy tham khảo kỹ các hướng dẫn chính thức của Google Play Console và App Store Connect để đảm bảo bạn thực hiện đúng các bước.

5.6 Câu hỏi ôn tập

1. API là gì? Khái niệm REST cơ bản trong bối cảnh ứng dụng di động.
2. Phân biệt GET/POST và tình huống sử dụng.
3. Geolocation cần quyền gì? Nêu lưu ý về privacy.
4. Image Picker dùng để làm gì? Các quyền thường gặp trên Android/iOS.
5. AsyncStorage dùng để làm gì? Nêu ví dụ lưu token/setting.
6. Debug trong React Native: log, dev menu, remote debugging…
7. Các bước xuất bản ứng dụng Android (AAB/APK) gồm những gì?
8. Nêu rủi ro khi dùng thư viện đã lỗi thời (ví dụ CameraRoll cũ).

5.7 Bài tập thực hành

1. Gọi API public (ví dụ JSONPlaceholder) và hiển thị danh sách bài viết bằng FlatList.
2. Tạo chức năng lưu 'favorite' vào AsyncStorage và đọc lại khi mở app.
3. Dùng Image Picker chọn ảnh và hiển thị preview (kèm xử lý quyền).
4. Viết checklist build release Android và tạo icon/splash cơ bản.

5.8 Phiếu bài tập thực hành (Mẫu 04)

Sinh viên thực hiện theo yêu cầu, chụp màn hình/ghi video demo và nộp mã nguồn theo hướng dẫn của giảng viên.

Tên bài thực hành	Chương 5 – Lab 05: API + Lưu trữ cục bộ + Tài nguyên thiết bị
Mục tiêu	Kết hợp gọi API, lưu AsyncStorage và truy cập ảnh để hoàn thiện mini-app.

Thời lượng gợi ý	180 phút
Chuẩn bị	Đã cài đặt thư viện cần thiết (axios hoặc fetch; image picker; async-storage). Thiết bị/emulator có quyền truy cập ảnh.
Nhiệm vụ thực hành	Tạo màn hình Posts: tải 20 bài viết từ API và hiển thị. Cho phép 'Star/Favorite' một bài viết và lưu vào AsyncStorage. Tạo màn hình Favorites để đọc danh sách đã lưu. Thêm chức năng chọn ảnh đại diện (Image Picker) và lưu uri tạm thời (AsyncStorage).
Sản phẩm nộp	Repo + video demo 60–90s (Posts → Favorite → Favorites → chọn ảnh). Ảnh chụp màn hình permissions (nếu có).
Tiêu chí đánh giá	Gọi API + hiển thị list (25%). Lưu/đọc AsyncStorage đúng (30%). Image Picker hoạt động + xử lý quyền (25%). Tổng thể app chạy ổn, code sạch (20%).